```
import torch
import random
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
import numpy as np
import cv2
import os
from PIL import Image
import time
import torch.nn.functional as F
import matplotlib.pyplot as plt
from torchvision.transforms.functional import center_crop
from torch.utils.data import random_split
```

```
# === Parameters ===
nx = 128                # Simulation resolution
nm_per_pixel = 1.0      # Physical size per pixel (nm)
NA = 1.35               # Numerical aperture
wavelength = 193.0      # nm (ArF laser)
threshold = 0.25        # Threshold for resist development
num_iter = 50
sharpness = 100
lr=0.0001
batch_size = 16
learning_rate = 0.0001
num_epochs = 50
weight_decay = 10**-5
val_p = torch.zeros(128,128)
# print(val_p)
```

```
# import torch
# import cv2
# import os
from google.colab import drive

# # Mount Google Drive
drive.mount('/content/drive')

# # Configuration
# device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# input_folder = '/content/drive/MyDrive/Litho/Lalith/sample_masks'
# output_folder = '/content/drive/MyDrive/processed_masks'


# # Create output directory if not exists
# os.makedirs(output_folder, exist_ok=True)

# for i in range(1, 1201):
#     # Load image
#     img_path = f'{input_folder}/sample_{i}.png'
#     if not os.path.exists(img_path):
#         print(f"Missing file: {img_path}")
#         continue

#     # === Image processing ===
#     mask_target_np = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
#     h, w = mask_target_np.shape

#     # Padding
#     size = max(h, w)
#     pad_h = (size - h) // 2
#     pad_w = (size - w) // 2
#     padded_img = cv2.copyMakeBorder(
#         mask_target_np, pad_h, size - h - pad_h, pad_w, size - w - pad_w,
#         borderType=cv2.BORDER_CONSTANT, value=0
#     )

#     # Resizing and normalization
#     resized_img = cv2.resize(padded_img, (nx, nx), interpolation=cv2.INTER_NEAREST)
#     resized_img_tensor = torch.tensor(resized_img, dtype=torch.float32, device=device)
#     resized_img_tensor = resized_img_tensor / 255.0
```

```
#     # Save processed tensor
#     output_path = f'{output_folder}/processed_sample_{i}.png'
#     cv2.imwrite(output_path, resized_img)

# print("Processing completed for all 1200 samples!")
```

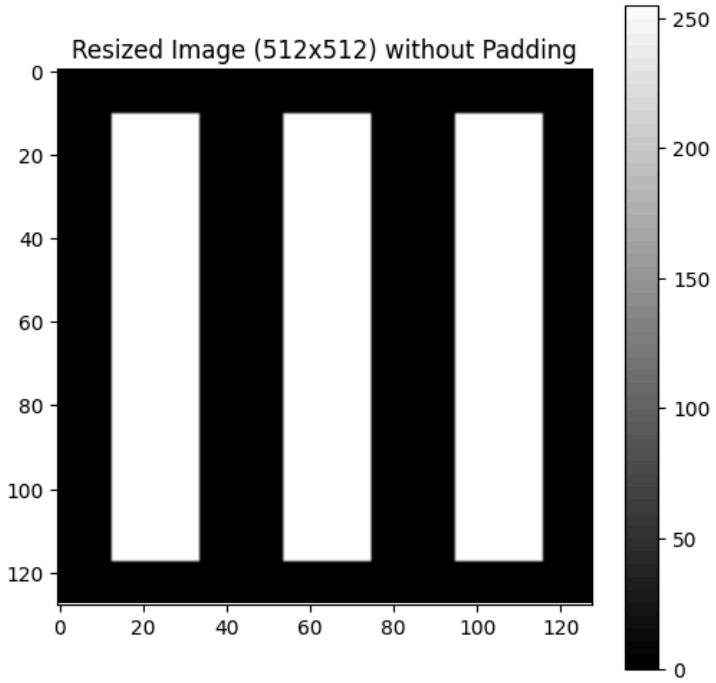
```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_
```

```
import torch
import torch.nn as nn

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class DoubleConv(nn.Module):
    """(convolution => [BN] => ReLU) * 2"""
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Dropout2d(0.2),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Dropout2d(0.2)
        )

    def forward(self, x):
        return self.double_conv(x)

class Down(nn.Module):
    """Downscaling with maxpool then double conv"""
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.maxpool_conv = nn.Sequential(
            nn.AvgPool2d(2),
            DoubleConv(in_channels, out_channels)
        )

    def forward(self, x):
        return self.maxpool_conv(x)

class Up(nn.Module):
    """Upscaling then double conv (no center crop)"""
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.up = nn.ConvTranspose2d(in_channels, in_channels // 2, kernel_size=2, stride=2)
        self.conv = DoubleConv(in_channels, out_channels)

    def forward(self, x1, x2):
        x1 = self.up(x1)
        # Assume x1 and x2 spatial sizes match; no cropping
        x = torch.cat([x2, x1], dim=1)
        return self.conv(x)

class OutConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=1)

    def forward(self, x):
        return self.conv(x)

class SmallUNet(nn.Module):
    def __init__(self, n_channels=1, n_classes=1):
        super().__init__()
        self.inc = DoubleConv(n_channels, 32)
        self.down1 = Down(32, 64)
        self.down2 = Down(64, 128)
        self.up1 = Up(128, 64)
        self.up2 = Up(64, 32)
        self.outc = OutConv(32, n_classes)
```

```
        self.sigmoid = nn.Tanh()

    # def forward(self, x):
    #     x1 = self.inc(x)
    #     x2 = self.down1(x1)
    #     x3 = self.down2(x2)
    #     x = self.up1(x3, x2)
    #     x = self.up2(x, x1)
    #     logits = self.outc(x)
    #     return logits

    def forward(self, x):
            x1 = self.inc(x)
            x2 = self.down1(x1)
            x3 = self.down2(x2)
            x = self.up1(x3, x2)
            x = self.up2(x, x1)
            logits = self.outc(x)
            return logits  # Remove the self.sigmoid(logits)


# Example usage:
# model = SmallUNet().to(device)
```

```
# new_image_path = '/content/ILT_line_space_120.png'
# new_mask_np = cv2.imread(new_image_path, cv2.IMREAD_GRAYSCALE)

# if new_mask_np is None:
#     print(f"Error: Could not load image from {new_image_path}")
# else:
#     print(f"Successfully loaded image with shape: {new_mask_np.shape}")

#     # === Padding and Resizing ===
#     H, W = new_mask_np.shape
#     siz = max(H, W)
#     pad_H = (siz - H) // 2
#     pad_W = (siz - W) // 2
#     padded_img = cv2.copyMakeBorder(
#         new_mask_np, pad_H, siz - H - pad_H, pad_W, siz - W - pad_W,
#         borderType=cv2.BORDER_CONSTANT, value=0
#     )
#     resized_img_512 = cv2.resize(padded_img, (512, 512), interpolation=cv2.INTER_NEAREST)

#     print(f"Padded and resized image shape: {resized_img_512.shape}")

#     # Display the resized image
#     plt.figure(figsize=(6, 6))
#     plt.imshow(resized_img_512, cmap='gray')
#     plt.title('Padded and Resized Image (512x512)')
#     plt.colorbar()
#     plt.show()

import cv2
import matplotlib.pyplot as plt

new_image_path = '/content/ILT_line_space_80.png'
new_mask_np = cv2.imread(new_image_path, cv2.IMREAD_GRAYSCALE)
H_g,W_g = new_mask_np.shape

if new_mask_np is None:
    print(f"Error: Could not load image from {new_image_path}")
else:
    print(f"Successfully loaded image with shape: {new_mask_np.shape}")

    # === Resizing to 512x512 without padding ===
    # Note: This will distort the aspect ratio if the original image is not square.
    resized_img_512 = cv2.resize(new_mask_np, (128, 128), interpolation=cv2.INTER_NEAREST)

    print(f"Resized image shape (without padding): {resized_img_512.shape}")

    # Display the resized image
    plt.figure(figsize=(6, 6))
    plt.imshow(resized_img_512, cmap='gray')
    plt.title('Resized Image (512x512) without Padding')
    plt.colorbar()
    plt.show()
```

```
Successfully loaded image with shape: (600, 500)
Resized image shape (without padding): (128, 128)
```




```
##MAIN##
def compute_aerial_image_torch(mask: torch.Tensor, NA, wavelength, nm_per_pixel,
                               ):
  #sharpness=40.0, threshold=0.3

    device = mask.device
    B, C, H, W = mask.shape   # ✅ Proper dimension handling

    fov_X = W * nm_per_pixel
    fov_Y = H * nm_per_pixel
    fov = max(fov_X, fov_Y)
    dx = dy = fov / nx


    fx = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dx, device=device)) # Ensure on device
    fy = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dy, device=device)) # Ensure on device

    FY, FX = torch.meshgrid(fy, fx, indexing='ij')
    FY = FY.to(device) # Ensure on device
    FX = FX.to(device) # Ensure on device
    # # Physical field of view
    # fov_x = W * nm_per_pixel
    # fov_y = H * nm_per_pixel

    # dx = nm_per_pixel
    # dy = nm_per_pixel

    # # Frequency coordinates (based on actual H and W!)
    # fx = torch.fft.fftshift(torch.fft.fftfreq(W, d=dx, device=device))
    # fy = torch.fft.fftshift(torch.fft.fftfreq(H, d=dy, device=device))
    # FY, FX = torch.meshgrid(fy, fx, indexing='ij')

    cutoff_freq = NA / wavelength
    pupil_filter = ((FX**2 + FY**2).sqrt() <= cutoff_freq).float()
    pupil_filter = pupil_filter[None, None, :, :]   # ✅ Broadcast over B and C

    # Convert to complex for FFT
    mask_complex = mask.to(torch.complex64)

    # FFT per mask in batch
    mask_ft = torch.fft.fftshift(torch.fft.fft2(mask_complex, dim=(-2, -1)))

    # Apply pupil filter
    filtered_ft = mask_ft * pupil_filter
```

```
    # Inverse FFT → aerial field
    field = torch.fft.ifft2(torch.fft.ifftshift(filtered_ft), dim=(-2, -1))

    # Aerial intensity
    aerial_image = torch.abs(field) ** 2

    # # Normalize PER MASK (not across batch!)
    # max_vals = aerial_image.amax(dim=(-2, -1), keepdim=True)
    # aerial_image = aerial_image / (max_vals + 1e-8)

    # Resist model
    resist_sigmoid = torch.sigmoid(sharpness * (aerial_image - threshold))

    return resist_sigmoid, aerial_image



def compute_aerial_image_torch_single(weighted_result, NA, wavelength, nm_per_pixel):
  # Ensure input is a tensor and on the correct device, preserving requires_grad
  if not isinstance(weighted_result, torch.Tensor):
      mask = torch.tensor(weighted_result, dtype=torch.float32, device=device)
  else:
      mask = weighted_result.to(device)

  # Normalize mask if needed, but keep the require_grad property
  if mask.max() > 0:
      mask = mask / mask.max()
  else:
      mask = mask # Avoid division by zero

  fov_X = W_g * nm_per_pixel
  fov_Y = H_g * nm_per_pixel
  fov = max(fov_X, fov_Y)
  dx = dy = fov / nx


  fx = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dx, device=device)) # Ensure on device
  fy = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dy, device=device)) # Ensure on device

  FY, FX = torch.meshgrid(fy, fx, indexing='ij')
  FY = FY.to(device) # Ensure on device
  FX = FX.to(device) # Ensure on device

  # Ensure mask is complex for fft
  mask_complex = mask.to(torch.complex64)

  mask_ft = torch.fft.fftshift(torch.fft.fft2(mask_complex))
  cutoff_freq = NA / wavelength
  pupil_filter = ((FX**2 + FY**2).sqrt() <= cutoff_freq).float()
  pupil_filter = pupil_filter.to(device) # Ensure on device
  filtered_ft = mask_ft * pupil_filter
  ifft_img = torch.fft.ifft2(torch.fft.ifftshift(filtered_ft))
  aerial_image = torch.abs(ifft_img) ** 2
  # ##TO REMOVE
  # # Normalize aerial image, but maintain require_grad
  # if aerial_image.max() > 0:
  #     aerial_image = aerial_image / aerial_image.max()
  # else:
  #     aerial_image = aerial_image # Avoid division by zero


  # Resist development using PyTorch operations
  resist_sigmoid = 1 / (1 + torch.exp(-sharpness * (aerial_image - threshold)))

  return resist_sigmoid, aerial_image # Return torch tensors
```

Aerial image code (B, C, H, W)

```
# def compute_aerial_image_torch(weighted_result: torch.Tensor,
#                                NA, wavelength, nm_per_pixel):
#     """
#     weighted_result: [B, 1, H, W]
#     returns:
#         resist_sigmoid: [B, 1, H, W]
```

```
#         aerial_image  : [B, 1, H, W]
#     """

#     # -------------------------
#     # Input
#     # -------------------------
#     mask = weighted_result.to(device)          # [B,1,H,W]
#     B, C, H, W = mask.shape
#     assert C == 1

#     # -------------------------
#     # Per-mask normalization (IMPORTANT FIX)
#     # -------------------------
#     mask_max = mask.amax(dim=(2, 3), keepdim=True)   # [B,1,1,1]
#     mask = torch.where(mask_max > 0, mask / mask_max, mask)

#     # -------------------------
#     # Field of view
#     # -------------------------
#     fov_x = W * nm_per_pixel
#     fov_y = H * nm_per_pixel
#     fov = max(fov_x, fov_y)

#     dx = dy = fov / H   # assume square grid

#     # -------------------------
#     # Frequency grid (shared optics)
#     # -------------------------
#     fx = torch.fft.fftshift(
#         torch.fft.fftfreq(W, d=dx, device=device)
#     )
#     fy = torch.fft.fftshift(
#         torch.fft.fftfreq(H, d=dy, device=device)
#     )

#     FY, FX = torch.meshgrid(fy, fx, indexing='ij')
#     FX = FX[None, None, :, :]   # [1,1,H,W]
#     FY = FY[None, None, :, :]

#     # -------------------------
#     # Fourier optics
#     # -------------------------
#     mask_complex = mask.to(torch.complex64)

#     mask_ft = torch.fft.fftshift(
#         torch.fft.fft2(mask_complex, dim=(-2, -1)),
#         dim=(-2, -1)
#     )

#     cutoff_freq = NA / wavelength
#     pupil_filter = ((FX**2 + FY**2).sqrt() <= cutoff_freq).float()

#     filtered_ft = mask_ft * pupil_filter

#     ifft_img = torch.fft.ifft2(
#         torch.fft.ifftshift(filtered_ft, dim=(-2, -1)),
#         dim=(-2, -1)
#     )

#     aerial_image = torch.abs(ifft_img) ** 2    # [B,1,H,W]

#     # -------------------------
#     # Per-mask aerial normalization (IMPORTANT FIX)
#     # -------------------------
#     aerial_max = aerial_image.amax(dim=(2, 3), keepdim=True)
#     aerial_image = torch.where(
#         aerial_max > 0, aerial_image / aerial_max, aerial_image
#     )

#     # -------------------------
#     # Resist model
#     # -------------------------
#     resist_sigmoid = torch.sigmoid(
#         sharpness * (aerial_image - threshold)
#     )
```

```
#     return resist_sigmoid, aerial_image
```

```
def mape(y_true, y_pred):
    mask = y_true != 0
    return (torch.abs((y_true[mask] - y_pred[mask]) / y_true[mask]).mean() * 100).item()

def rmspe(y_true, y_pred):
    mask = y_true != 0
    return ((((y_true[mask] - y_pred[mask]) / y_true[mask]) ** 2).mean()).sqrt() * 100

def epoch_cleanup():
    # Clear CUDA cache to free GPU memory
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    # Garbage collection for CPU tensors
    import gc
    gc.collect()
```

```
# def show_images(target, predicted, aerial_image, batch_idx=None, epoch=None):
#     # Convert tensors to numpy for visualization
#     # target_np = target.detach().cpu().squeeze().numpy()
#     # predicted_np = predicted.detach().cpu().squeeze().numpy()
#     # aerial_image_np = aerial_image.detach().cpu().squeeze().numpy()
#     target_np = target.cpu().numpy()
#     predicted_np = predicted.cpu().numpy()
#     aerial_image_np = aerial_image.cpu().numpy()

#     # Plotting
#     plt.figure(figsize=(10, 4))
#     plt.subplot(1, 3, 1)
#     plt.title('Target Layout')
#     plt.imshow(target_np, cmap='gray')
#     plt.axis('off')

#     plt.subplot(1, 3, 2)
#     plt.title('Predicted Mask')
#     plt.imshow(predicted_np, cmap='gray')
#     plt.axis('off')

#     plt.subplot(1, 3, 3)
#     plt.title('Aerial Image of Predicted Mask')
#     plt.imshow(aerial_image_np, cmap='gray')
#     plt.axis('off')

#     if batch_idx is not None and epoch is not None:
#         plt.suptitle(f'Epoch {epoch+1}, Batch {batch_idx}')
#     elif batch_idx is not None:
#         plt.suptitle(f'Batch {batch_idx}')
#     plt.tight_layout()
#     plt.show()



# def train_physics_informed():
#     class LayoutDataset(Dataset):
#         def __init__(self, layout_dir, transform=None):
#             self.layout_dir = layout_dir
#             self.transform = transform
#             self.layouts = os.listdir(layout_dir)

#         def __len__(self):
#             return len(self.layouts)

#         def __getitem__(self, idx):
#             img_path = os.path.join(self.layout_dir, self.layouts[idx])
#             img = Image.open(img_path).convert('L')
#             if self.transform:
#                 img = self.transform(img)
#             return img
#     class SingleImageDataset(Dataset):
#         def __init__(self, image_path, transform=None):
#             self.image_path = image_path
```

```
#             self.transform = transform

#         def __len__(self):
#             return 1

#         def __getitem__(self, idx):
#             img = Image.open(self.image_path).convert('L')
#             if self.transform:
#                 img = self.transform(img)
#             return img
#     # Data loading
#     dataset = LayoutDataset(
#         layout_dir='/content/drive/MyDrive/processed_masks',
#         transform = transforms.Compose([
#         transforms.Resize((nx, nx)),
#         transforms.ToTensor(),  # Converts to tensor and normalizes to [0,1]
#     ])
#     )
#     # Calculate split lengths
#     total_size = len(dataset)
#     train_size = int(0.8 * total_size)
#     val_size = total_size - train_size

#     train_dataset, val_dataset = random_split(dataset, [train_size, val_size])


#     train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0)

#     # === SINGLE IMAGE for validation ===
#     val_image_path = '/content/ILT_line_space_120.png'
#     val_dataset = SingleImageDataset(
#         image_path=val_image_path,
#         transform=transforms.Compose([
#             transforms.Resize((nx, nx)),
#             transforms.ToTensor(),
#         ])
#     )
#     val_loader = DataLoader(val_dataset, batch_size=1, shuffle=False)

#     # val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=0)
#     model = SmallUNet().to(device)

#     optimizer = optim.Adam(model.parameters(), lr=learning_rate, weight_decay = weight_decay)
#     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode = 'min', patience=5)

#     # Start timing
#     train_start_time = time.time()

#     loss_history = {
#     'train_mse': [],
#     'train_mae': [],
#     'val_mse': [],
#     'val_mae': [],
#     'train_mean_error': [],
#     'val_mean_error': []
# }


#     for epoch in range(num_epochs):
#         model.train()
#         running_mae = 0.0
#         running_mse = 0.0
#         running_mape = 0.0
#         running_rmspe = 0.0
#         running_mean_error = 0.0
#         n_samples = 0

#         for batch_idx, target_layouts in enumerate(train_loader):
#             target_layouts = target_layouts.float().to(device)
#             if target_layouts.dim() == 3:
#                 target_layouts = target_layouts.unsqueeze(1)  # Ensure [B, 1, H, W]

#             # Forward pass
#             predicted_masks = model(target_layouts)

#             # Physics simulation
```

```
#             _, aerial_images = compute_aerial_image_torch(
#                 predicted_masks, NA, wavelength, nm_per_pixel)

#             # print(aerial_images.shape)

#             batch_mape = mape(target_layouts, aerial_images)
#             batch_rmspe = rmspe(target_layouts, aerial_images)

#             mean_error_loss = torch.abs(torch.mean(aerial_images - target_layouts))
#             mse = F.mse_loss(aerial_images, target_layouts)
#             mae = F.l1_loss(aerial_images, target_layouts)

#             # Backpropagation
#             optimizer.zero_grad()
#             mse.backward()
#             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
#             optimizer.step()

#             running_mape += batch_mape * target_layouts.size(0)
#             running_rmspe += batch_rmspe * target_layouts.size(0)
#             running_mean_error += mean_error_loss.item() * target_layouts.size(0)
#             running_mse += mse.item() * target_layouts.size(0)
#             running_mae += mae.item() * target_layouts.size(0)
#             n_samples += target_layouts.size(0)


#             if batch_idx % 10 == 0:
#                 print(f'Epoch [{epoch+1}/{num_epochs}], Step [{batch_idx}/{len(train_loader)}], '
#                       f'Batch MSE: {mse.item():.6f}, Batch MAE: {mae.item():.6f}')


#         # Compute mean errors for the epoch
#         epoch_mse = running_mse / n_samples
#         epoch_mape = running_mape / n_samples
#         epoch_rmspe = running_rmspe / n_samples
#         epoch_mae = running_mae / n_samples
#         epoch_mean_error = running_mean_error / n_samples

#         # --- Save loss history ---
#         loss_history['train_mse'].append(epoch_mse)
#         loss_history['train_mae'].append(epoch_mae)
#         loss_history['train_mean_error'].append(epoch_mean_error)

#         print(f'Epoch [{epoch+1}/{num_epochs}] completed. Train MSE: {epoch_mse:.6f}, RMSPE%: {epoch_rmspe:.2f}
#         epoch_cleanup()

#     # ---- VALIDATION LOOP FOR SINGLE IMAGE ----
#     model.eval()
#     with torch.no_grad():
#         # Get the single validation batch
#         val_batch = next(iter(val_loader))
#         val_layouts = val_batch.float().to(device)
#         if val_layouts.dim() == 3:
#             val_layouts = val_layouts.unsqueeze(1)  # Now [1,1,H,W]

#         val_preds_4d = model(val_layouts)  # Model output: [1,1,H,W]

#         # Squeeze to 2D for single-image metrics and comparison
#         val_preds_2d = val_preds_4d.squeeze()
#         val_layouts_2d = val_layouts.squeeze()
#         # # -----------------------------
#         # # Convert prediction to 2D safely
#         # # -----------------------------
#         # print("val_preds_4d shape BEFORE reshape:", val_preds_4d.shape)

#         # if val_preds_4d.dim() == 4:          # [B,C,H,W]
#         #     val_preds_2d = val_preds_4d[0, 0]  # take first image, first channel
#         # elif val_preds_4d.dim() == 3:        # [C,H,W]
#         #     val_preds_2d = val_preds_4d[0]
#         # elif val_preds_4d.dim() == 2:        # already [H,W]
#         #     val_preds_2d = val_preds_4d
#         # else:
#         #     raise ValueError("Unexpected val_preds_4d shape")

#         # print("val_preds_2d shape AFTER reshape:", val_preds_2d.shape)
```

```
#         # # ----------------------------
#         # # Do the same for ground truth
#         # # ----------------------------
#         # print("val_layouts shape BEFORE reshape:", val_layouts.shape)

#         # if val_layouts.dim() == 4:           # [B,C,H,W]
#         #     val_layouts_2d = val_layouts[0, 0]
#         # elif val_layouts.dim() == 3:         # [C,H,W]
#         #     val_layouts_2d = val_layouts[0]
#         # elif val_layouts.dim() == 2:
#         #     val_layouts_2d = val_layouts
#         # else:
#         #     raise ValueError("Unexpected val_layouts shape")

#         # print("val_layouts_2d shape AFTER reshape:", val_layouts_2d.shape)

#         global val_p
#         val_p = val_preds_2d
#         # print("Pred Dimensions", val_preds_2d.shape)

#         _, val_aerial_2d = compute_aerial_image_torch_single(
#             val_preds_2d, NA, wavelength, nm_per_pixel)

#         # Calculate metrics using 2D tensors
#         val_loss = F.mse_loss(val_aerial_2d, val_layouts_2d).item()
#         val_mae = F.l1_loss(val_aerial_2d, val_layouts_2d).item()
#         val_mean_error = torch.mean(val_aerial_2d - val_layouts_2d).item()

#         # MAPE and RMSPE for the single batch
#         val_mape = mape(val_layouts_2d, val_aerial_2d)
#         val_rmspe = rmspe(val_layouts_2d, val_aerial_2d)

#         loss_history['val_mse'].append(val_loss)
#         loss_history['val_mae'].append(val_mae)
#         loss_history['val_mean_error'].append(val_mean_error)

#     # ---- SCHEDULER STEP WITH VALIDATION LOSS ----
#     scheduler.step(val_loss)

#     # Display the single validation image using 2D tensors
#     show_images(val_layouts_2d, val_preds_2d, val_aerial_2d, epoch=epoch)

#     print(f'Validation MSE: {val_loss:.6f}, RMSE%: {val_rmspe:.2f}%')
#     print(f'Validation MAE: {val_mae:.6f}, MAE%: {val_mape:.2f}%')

#     total_time = time.time() - train_start_time
#     print(f"Training completed in {total_time/60:.2f} minutes.")

#     torch.save(model.state_dict(), 'physics_informed_unet.pth')
#     import pickle
#     with open('loss_history.pkl', 'wb') as f:
#         pickle.dump(loss_history, f)
#     return loss_history

# if __name__ == '__main__':
#     loss_history = train_physics_informed()
```

MAIN

```
def show_images(target, predicted, aerial_image, batch_idx=None, epoch=None):
    # Convert tensors to numpy for visualization
    # squeeze() handles 4D [1,1,H,W] or 3D [1,H,W] tensors correctly to [H,W]
    target_np = target.detach().cpu().squeeze().numpy()
    predicted_np = predicted.detach().cpu().squeeze().numpy()
    aerial_image_np = aerial_image.detach().cpu().squeeze().numpy()

    # Plotting
    plt.figure(figsize=(10, 4))
    plt.subplot(1, 3, 1)
    plt.title('Target Layout')
    plt.imshow(target_np, cmap='gray')
    plt.axis('off')

    plt.subplot(1, 3, 2)
```

```
    plt.title('Predicted Mask')
    plt.imshow(predicted_np, cmap='gray')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.title('Aerial Image')
    plt.imshow(aerial_image_np, cmap='gray')
    plt.axis('off')

    if batch_idx is not None and epoch is not None:
        plt.suptitle(f'Epoch {epoch+1}, Batch {batch_idx}')
    elif batch_idx is not None:
        plt.suptitle(f'Batch {batch_idx}')
    elif epoch is not None:
        plt.suptitle(f'Epoch {epoch+1}')
    plt.tight_layout()
    plt.show()


def train_physics_informed():
    class LayoutDataset(Dataset):
        def __init__(self, layout_dir, transform=None):
            self.layout_dir = layout_dir
            self.transform = transform
            self.layouts = os.listdir(layout_dir)

        def __len__(self):
            return len(self.layouts)

        def __getitem__(self, idx):
            img_path = os.path.join(self.layout_dir, self.layouts[idx])
            img = Image.open(img_path).convert('L')
            if self.transform:
                img = self.transform(img)
            return img

    class SingleImageDataset(Dataset):
        def __init__(self, image_path, transform=None):
            self.image_path = image_path
            self.transform = transform

        def __len__(self):
            return 1

        def __getitem__(self, idx):
            img = Image.open(self.image_path).convert('L')
            if self.transform:
                img = self.transform(img)
            return img

    # Data loading
    dataset = LayoutDataset(
        layout_dir='/content/drive/MyDrive/processed_masks',
        transform = transforms.Compose([
        transforms.Resize((nx, nx)),
        transforms.ToTensor(),
    ])
    )

    total_size = len(dataset)
    train_size = int(0.8 * total_size)
    val_size = total_size - train_size
    train_dataset, _ = random_split(dataset, [train_size, val_size])
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0)

    # === SINGLE IMAGE for validation ===
    val_image_path = '/content/ILT_line_space_80.png'
    val_dataset = SingleImageDataset(
        image_path=val_image_path,
        transform=transforms.Compose([
            transforms.Resize((nx, nx)),
            transforms.ToTensor(),
        ])
    )
    val_loader = DataLoader(val_dataset, batch_size=1, shuffle=False)
```

```
model = SmallUNet().to(device)

optimizer = optim.Adam(model.parameters(), lr=learning_rate, weight_decay=weight_decay)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=5)

train_start_time = time.time()

loss_history = {
    'train_mse': [],
    'train_mae': [],
    'val_mse': [],
    'val_mae': [],
    'train_mean_error': [],
    'val_mean_error': []
}

for epoch in range(num_epochs):
    model.train()
    running_mae = 0.0
    running_mse = 0.0
    running_mape = 0.0
    running_rmspe = 0.0
    running_mean_error = 0.0
    n_samples = 0

    for batch_idx, target_layouts in enumerate(train_loader):
        target_layouts = target_layouts.float().to(device)
        if target_layouts.dim() == 3:
            target_layouts = target_layouts.unsqueeze(1)

        predicted_masks = model(target_layouts)

        # Physics simulation
        aerial_images, _ = compute_aerial_image_torch(
            predicted_masks, NA, wavelength, nm_per_pixel)

        batch_mape = mape(target_layouts, aerial_images)
        batch_rmspe = rmspe(target_layouts, aerial_images)

        mean_error_loss = torch.abs(torch.mean(aerial_images - target_layouts))
        mse = F.mse_loss(aerial_images, target_layouts)
        mae = F.l1_loss(aerial_images, target_layouts)

        optimizer.zero_grad()
        mse.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

        running_mape += batch_mape * target_layouts.size(0)
        running_rmspe += batch_rmspe * target_layouts.size(0)
        running_mean_error += mean_error_loss.item() * target_layouts.size(0)
        running_mse += mse.item() * target_layouts.size(0)
        running_mae += mae.item() * target_layouts.size(0)
        n_samples += target_layouts.size(0)

        if batch_idx % 20 == 0:
            print(f'Epoch [{epoch+1}/{num_epochs}], Step [{batch_idx}/{len(train_loader)}], ' +
                  f'Batch MSE: {mse.item():.6f}, Batch MAE: {mae.item():.6f}')

    epoch_mse = running_mse / n_samples
    loss_history['train_mse'].append(epoch_mse)
    loss_history['train_mae'].append(running_mae / n_samples)
    loss_history['train_mean_error'].append(running_mean_error / n_samples)

    print(f'Epoch [{epoch+1}/{num_epochs}] completed. Train MSE: {epoch_mse:.6f}')
    epoch_cleanup()

# ---- VALIDATION LOOP FOR SINGLE IMAGE ----
model.eval()
with torch.no_grad():
    val_batch = next(iter(val_loader))
    val_layouts = val_batch.float().to(device)
    if val_layouts.dim() == 3:
        val_layouts = val_layouts.unsqueeze(1)

    val_preds_4d = model(val_layouts)
```

```
            # Use batch version of simulation to maintain consistent dimensions
            val_aerial_4d, _ = compute_aerial_image_torch_single(
                val_preds_4d, NA, wavelength, nm_per_pixel)

            val_loss = F.mse_loss(val_aerial_4d, val_layouts).item()
            val_mae = F.l1_loss(val_aerial_4d, val_layouts).item()
            val_mean_error = torch.mean(val_aerial_4d - val_layouts).item()

            # metrics on matching dimensions
            val_mape = mape(val_layouts, val_aerial_4d)
            val_rmspe = rmspe(val_layouts, val_aerial_4d)

            loss_history['val_mse'].append(val_loss)
            loss_history['val_mae'].append(val_mae)
            loss_history['val_mean_error'].append(val_mean_error)

            # Update global val_p for optimization steps in other cells
            global val_p
            val_p = val_preds_4d[0].squeeze().detach().cpu()

        scheduler.step(val_loss)

        # Display images
        show_images(val_layouts, val_preds_4d, val_aerial_4d, epoch=epoch)

        print(f'Validation MSE: {val_loss:.6f}, RMSE%: {val_rmspe:.2f}%')
        print(f'Validation MAE: {val_mae:.6f}, MAE%: {val_mape:.2f}%')
        print(f'Validation Mean Error: {val_mean_error:.6f}')

        total_time = time.time() - train_start_time
        print(f"Training completed in {total_time/60:.2f} minutes.")

        torch.save(model.state_dict(), 'physics_informed_unet.pth')
        with open('loss_history.pkl', 'wb') as f:
            import pickle
            pickle.dump(loss_history, f)
        return loss_history

    if __name__ == '__main__':
        loss_history = train_physics_informed()
```

```
Epoch [1/50], Step [0/60], Batch MSE: 0.302063, Batch MAE: 0.343623
Epoch [1/50], Step [20/60], Batch MSE: 0.163741, Batch MAE: 0.170963
Epoch [1/50], Step [40/60], Batch MSE: 0.227408, Batch MAE: 0.238455
Epoch [1/50] completed. Train MSE: 0.214485
Epoch [2/50], Step [0/60], Batch MSE: 0.222800, Batch MAE: 0.229490
Epoch [2/50], Step [20/60], Batch MSE: 0.164238, Batch MAE: 0.170654
Epoch [2/50], Step [40/60], Batch MSE: 0.209724, Batch MAE: 0.217523
Epoch [2/50] completed. Train MSE: 0.180381
Epoch [3/50], Step [0/60], Batch MSE: 0.164504, Batch MAE: 0.170560
Epoch [3/50], Step [20/60], Batch MSE: 0.190335, Batch MAE: 0.196904
Epoch [3/50], Step [40/60], Batch MSE: 0.161712, Batch MAE: 0.182156
Epoch [3/50] completed. Train MSE: 0.174363
Epoch [4/50], Step [0/60], Batch MSE: 0.229716, Batch MAE: 0.237587
Epoch [4/50], Step [20/60], Batch MSE: 0.118545, Batch MAE: 0.124093
Epoch [4/50], Step [40/60], Batch MSE: 0.193413, Batch MAE: 0.200616
Epoch [4/50] completed. Train MSE: 0.173303
Epoch [5/50], Step [0/60], Batch MSE: 0.109200, Batch MAE: 0.114405
Epoch [5/50], Step [20/60], Batch MSE: 0.150755, Batch MAE: 0.164202
Epoch [5/50], Step [40/60], Batch MSE: 0.165257, Batch MAE: 0.170910
Epoch [5/50] completed. Train MSE: 0.171267
Epoch [6/50], Step [0/60], Batch MSE: 0.177462, Batch MAE: 0.198809
Epoch [6/50], Step [20/60], Batch MSE: 0.204236, Batch MAE: 0.211347
Epoch [6/50], Step [40/60], Batch MSE: 0.148806, Batch MAE: 0.154989
Epoch [6/50] completed. Train MSE: 0.170459
Epoch [7/50], Step [0/60], Batch MSE: 0.154327, Batch MAE: 0.160528
Epoch [7/50], Step [20/60], Batch MSE: 0.133671, Batch MAE: 0.139187
Epoch [7/50], Step [40/60], Batch MSE: 0.114440, Batch MAE: 0.119640
Epoch [7/50] completed. Train MSE: 0.172986
Epoch [8/50], Step [0/60], Batch MSE: 0.183725, Batch MAE: 0.194816
Epoch [8/50], Step [20/60], Batch MSE: 0.227691, Batch MAE: 0.235089
Epoch [8/50], Step [40/60], Batch MSE: 0.155815, Batch MAE: 0.161902
Epoch [8/50] completed. Train MSE: 0.173331
Epoch [9/50], Step [0/60], Batch MSE: 0.203526, Batch MAE: 0.210672
Epoch [9/50], Step [20/60], Batch MSE: 0.183174, Batch MAE: 0.189519
Epoch [9/50], Step [40/60], Batch MSE: 0.165914, Batch MAE: 0.172113
Epoch [9/50] completed. Train MSE: 0.173795
Epoch [10/50], Step [0/60], Batch MSE: 0.152591, Batch MAE: 0.158573
Epoch [10/50], Step [20/60], Batch MSE: 0.209058, Batch MAE: 0.219754
Epoch [10/50], Step [40/60], Batch MSE: 0.155381, Batch MAE: 0.161679
Epoch [10/50] completed. Train MSE: 0.168345
Epoch [11/50], Step [0/60], Batch MSE: 0.125892, Batch MAE: 0.131502
Epoch [11/50], Step [20/60], Batch MSE: 0.166724, Batch MAE: 0.172862
Epoch [11/50], Step [40/60], Batch MSE: 0.164916, Batch MAE: 0.171175
Epoch [11/50] completed. Train MSE: 0.168316
Epoch [12/50], Step [0/60], Batch MSE: 0.155142, Batch MAE: 0.161456
Epoch [12/50], Step [20/60], Batch MSE: 0.154531, Batch MAE: 0.163142
Epoch [12/50], Step [40/60], Batch MSE: 0.174597, Batch MAE: 0.181237
Epoch [12/50] completed. Train MSE: 0.170885
Epoch [13/50], Step [0/60], Batch MSE: 0.171052, Batch MAE: 0.177514
Epoch [13/50], Step [20/60], Batch MSE: 0.188399, Batch MAE: 0.194876
Epoch [13/50], Step [40/60], Batch MSE: 0.167479, Batch MAE: 0.174050
Epoch [13/50] completed. Train MSE: 0.167778
Epoch [14/50], Step [0/60], Batch MSE: 0.171082, Batch MAE: 0.177423
Epoch [14/50], Step [20/60], Batch MSE: 0.165079, Batch MAE: 0.171496
Epoch [14/50], Step [40/60], Batch MSE: 0.228056, Batch MAE: 0.235291
Epoch [14/50] completed. Train MSE: 0.168331
Epoch [15/50], Step [0/60], Batch MSE: 0.191132, Batch MAE: 0.197896
```

```
def h_to_mask(h):
  return 0.5 * (1 + np.cos(h))

def dh_dm(h):
  return -0.5 * np.sin(h)
```

```
Epoch [17/50], Step [0/60], Batch MSE: 0.167294, Batch MAE: 0.173398
```

```
# h=np.arccos(2.0 * )
```

```
Epoch [17/50] completed. Train MSE: 0.170017
```

```
# Enchancement 2 - Discretization penalty 3.2.1
def discretization_penalty(mask):
    # return np.sum(4 * mask * (1 - mask))
    return np.sum(4 * mask * (1 - mask))

def discretization_gradient(mask):
    return 4 * (1 - 2 * mask)
```

```
Epoch [20/50], Step [0/60], Batch MSE: 0.179964, Batch MAE: 0.185747
```

```
import torch

print("\n--- Discretization Optimization (Using Provided Functions) ---")

# Start from predicted mask
```

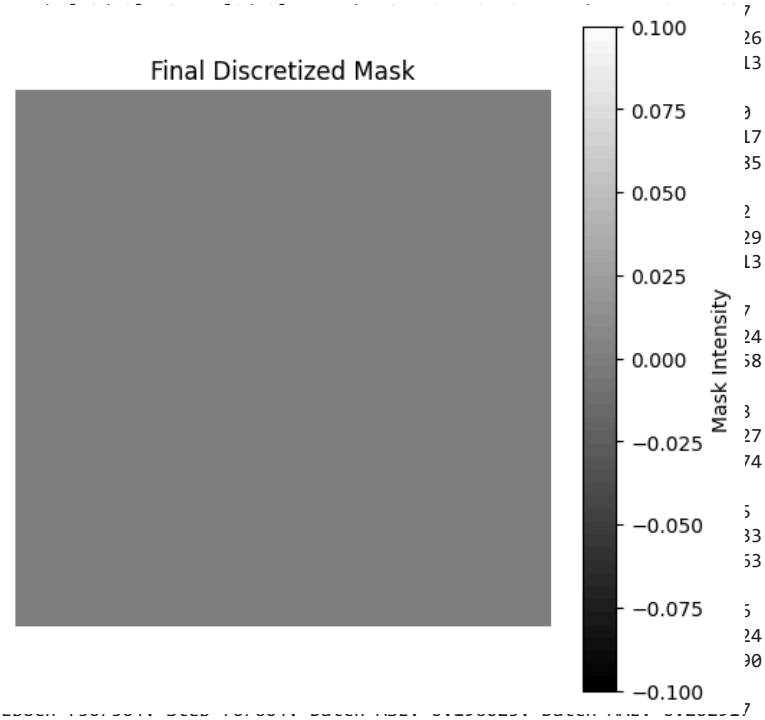
```
# m = val_p.clone().detach().cpu().numpy().to(device).requires_grad_(False)
print(val_p)
m = val_p.cpu().numpy()
print(m)
cdis = 0.005
step = 3
iters = 80

for itr in range(iters):

    # ----- Penalty -----
    Rd = discretization_penalty(m)
    total_cost = cdis * Rd

    # ----- Gradient -----
    grad_m = cdis * discretization_gradient(m)

    # ----- Gradient Descent Update -----
    with torch.no_grad():
        m -= step * grad_m
        m = np.clip(m, 0.0, 1.0)

    if itr % 5 == 0:
        print(f"[DIS] Iter {itr:03d} | Rd = {Rd.item():.4e}")

print("Discretization complete.")
```

```
Epoch [29/50], Step [40/60], Batch MSE: 0.133959, Batch MAE: 0.139022
Epoch [29/50] completed. Train MSE: 0.169839
--- Discretization Optimization (Using Provided Functions) ---
tensor([[0.2157, 0.2249, 0.2405, ..., 0.2289, 0.2251, 0.2051],
Epoch [30/50], Step [0/60], Batch MSE: 0.190544, Batch MAE: 0.206088
        [0.2151, 0.2339, 0.2353, ..., 0.2478, 0.2315, 0.2059],
Epoch [30/50], Step [20/60], Batch MSE: 0.148877, Batch MAE: 0.155086
        [0.2496, 0.2569, 0.2671, ..., 0.2603, 0.2428, 0.2161],
Epoch [30/50], Step [40/60], Batch MSE: 0.131261, Batch MAE: 0.126794
Epoch [30/50] completed. Train MSE: 0.169868
        ...,
Epoch [31/50], Step [0/60], Batch MSE: 0.104822, Batch MAE: 0.109870
        [0.2169, 0.2371, 0.2583, ..., 0.2400, 0.2357, 0.2036],
Epoch [31/50], Step [20/60], Batch MSE: 0.151237, Batch MAE: 0.157014
        [0.2339, 0.2583, 0.2532, ..., 0.2596, 0.2397, 0.2193],
Epoch [31/50], Step [40/60], Batch MSE: 0.158584, Batch MAE: 0.165317
        [0.2317, 0.2537, 0.2410, ..., 0.2319, 0.2426, 0.2025]])
Epoch [31/50] completed. Train MSE: 0.170075
[[0.21566448 0.22485   0.24087415 ... 0.22830359 0.2230797  0.20511496]
Epoch [32/50], Step [0/60], Batch MSE: 0.129607, Batch MAE: 0.209841
 [0.21530625 0.23903121 0.23483853 ... 0.24700027 0.21145572 0.20587417]
Epoch [32/50], Step [20/60], Batch MSE: 0.189134, Batch MAE: 0.195454
 [0.24935333 0.25604805 0.26723797 ... 0.26048785 0.24281526 0.21013617]
Epoch [32/50], Step [40/60], Batch MSE: 0.208943, Batch MAE: 0.216546
 ...
Epoch [32/50] completed. Train MSE: 0.167846
 [0.21686529 0.23712558 0.23883204 ... 0.239971   0.23968387 0.20363986]
Epoch [33/50], Step [0/60], Batch MSE: 0.156929, Batch MAE: 0.169251
 [0.23398682 0.25847506 0.25325584 ... 0.25197415 0.21971115 0.21331972]
Epoch [33/50], Step [20/60], Batch MSE: 0.198832, Batch MAE: 0.215038
 [0.23188534 0.23966651 0.24137037 ... 0.23187248 0.24260054 0.20249502]]
Epoch [33/50], Step [40/60], Batch MSE: 0.139436, Batch MAE: 0.145192
Epoch [33/50] completed. Train MSE: 0.169873
Epoch [34/50], Step [0/60], Batch MSE: 0.115151, Batch MAE: 0.120486
Epoch [34/50], Step [20/60], Batch MSE: 0.249210, Batch MAE: 0.256890
Epoch [34/50], Step [40/60], Batch MSE: 0.163690, Batch MAE: 0.169773
Epoch [34/50] completed. Train MSE: 0.168979
Epoch [35/50], Step [0/60], Batch MSE: 0.202403, Batch MAE: 0.209182
Epoch [35/50], Step [20/60], Batch MSE: 0.106879, Batch MAE: 0.111695
Epoch [35/50], Step [40/60], Batch MSE: 0.167940, Batch MAE: 0.174125
Epoch [35/50] completed. Train MSE: 0.168934
Epoch [36/50], Step [0/60], Batch MSE: 0.142864, Batch MAE: 0.148848
Epoch [36/50], Step [20/60], Batch MSE: 0.138901, Batch MAE: 0.144824
Epoch [36/50], Step [40/60], Batch MSE: 0.168918, Batch MAE: 0.174803
Epoch [36/50] completed. Train MSE: 0.168460
Epoch [37/50], Step [0/60], Batch MSE: 0.204898, Batch MAE: 0.212066
Epoch [37/50], Step [20/60], Batch MSE: 0.173491, Batch MAE: 0.179464
Epoch [37/50], Step [40/60], Batch MSE: 0.182590, Batch MAE: 0.189368
Epoch [37/50] completed. Train MSE: 0.168217
```

```
[DIS] Iter 000 | Rd = 4.0134e+04
[DIS] Iter 005 | Rd = 1.7178e+03
[DIS] Iter 010 | Rd = 0.0000e+00
[DIS] Iter 015 | Rd = 0.0000e+00
[DIS] Iter 020 | Rd = 0.0000e+00
[DIS] Iter 025 | Rd = 0.0000e+00
[DIS] Iter 030 | Rd = 0.0000e+00
[DIS] Iter 035 | Rd = 0.0000e+00
[DIS] Iter 040 | Rd = 0.0000e+00
[DIS] Iter 045 | Rd = 0.0000e+00
[DIS] Iter 050 | Rd = 0.0000e+00
[DIS] Iter 055 | Rd = 0.0000e+00
[DIS] Iter 060 | Rd = 0.0000e+00
[DIS] Iter 065 | Rd = 0.0000e+00
[DIS] Iter 070 | Rd = 0.0000e+00
[DIS] Iter 075 | Rd = 0.0000e+00
Discretization complete.
```

```
import matplotlib.pyplot as plt

# Detach from graph and move to CPU
final_mask = m
# print(final_mask.shape)
# Remove channel dimension if present
final_mask = final_mask.squeeze()

plt.figure(figsize=(6,6))
plt.imshow(final_mask, cmap='gray')
plt.title("Final Discretized Mask")
plt.colorbar(label="Mask Intensity")
plt.axis('off')
plt.show()
```

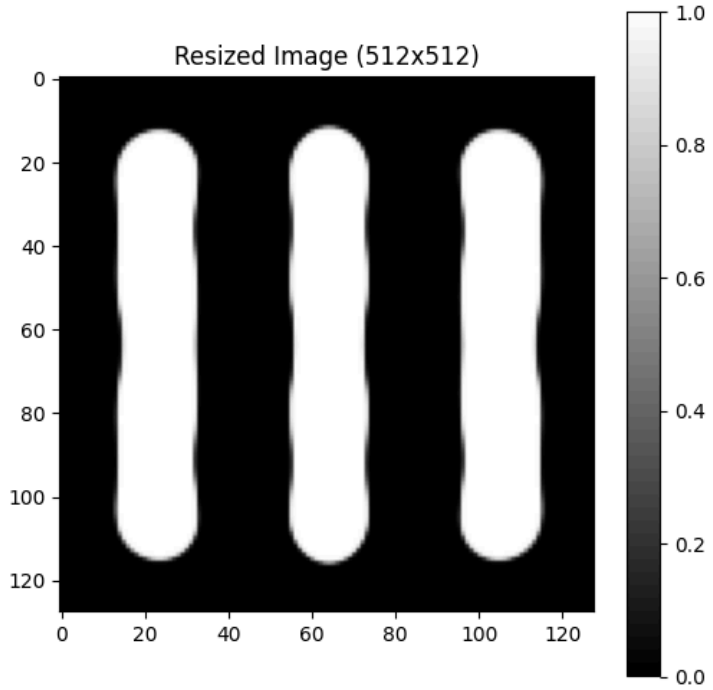
```
Epoch [42/50], Step [40/60], Batch MSE: 0.197858, Batch MAE: 0.204851
Epoch [42/50] completed. Train MSE: 0.168207
```




```
Epoch [50/50], Step [20/60], Batch MSE: 0.172820, Batch MAE: 0.184790
```

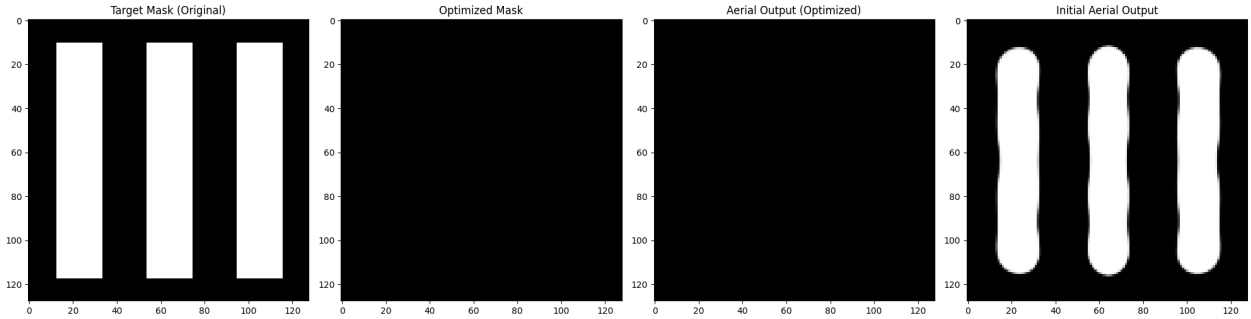
```
def compute_aerial_img(weighted_result, NA, wavelength, nm_per_pixel):
    # Ensure input is a tensor and on the correct device
    if not isinstance(weighted_result, torch.Tensor):
        mask = torch.tensor(weighted_result, dtype=torch.float32, device=device)
    else:
        mask = weighted_result.to(device)
    # Explicitly normalize if input looks like 0-255 uint8 data
    if mask.max() > 1.0:
        mask = mask / 255.0

    # H, W = mask.shape
    # print(H,W)
    fov_X = W_g * nm_per_pixel
    fov_Y = H_g * nm_per_pixel
    fov = max(fov_X, fov_Y)
    dx = dy = fov / nx


    fx = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dx, device=device)) # Ensure on device
    fy = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dy, device=device)) # Ensure on device

    FY, FX = torch.meshgrid(fy, fx, indexing='ij')
    FY = FY.to(device) # Ensure on device
    FX = FX.to(device) # Ensure on device
    # Ensure mask is complex for fft
    # print(FX,FY)
    mask_complex = mask.to(torch.complex64)
    mask_ft = torch.fft.fftshift(torch.fft.fft2(mask_complex))
    cutoff_freq = NA / wavelength
    pupil_filter = ((FX**2 + FY**2).sqrt() <= cutoff_freq)
    pupil_filter = pupil_filter.to(device) # Ensure on device
    #with np.printoptions(threshold=np.inf, linewidth=200):
     #   print(pupil_filter.cpu().numpy())
    filtered_ft = mask_ft * pupil_filter
    ifft_img = torch.fft.ifft2(torch.fft.ifftshift(filtered_ft))
    aerial_image = torch.abs(ifft_img) ** 2
    # print(cutoff_freq)
    # print(pupil_filter)
    # print(filtered_ft)
    # Robust normalization for visualization
    am_min = aerial_image.min()
    am_max = aerial_image.max()
    if am_max > am_min:
        aerial_image = (aerial_image - am_min) / (am_max - am_min)

    resist_sigmoid = torch.sigmoid(sharpness * (aerial_image - threshold))

    return resist_sigmoid, aerial_image
```

```
resist_initial_tensor, aerial_initial_tensor= compute_aerial_img(resized_img_512, NA, wavelength, nm_per_pixel)
# with np.printoptions(threshold=np.inf, linewidth=200):
#     print(resized_img_512)
plt.figure(figsize=(6, 6))
plt.imshow((resist_initial_tensor).cpu().numpy(), cmap='gray')
plt.title('Resized Image (512x512)')
plt.colorbar()
plt.show()
```




```
# Start timer
start_time = time.time()

resized_img_512_tensor = torch.tensor(resized_img_512)
# Ensure target is a normalized tensor [0, 1]
# if isinstance(resized_img_512, np.ndarray):
#     target_val = resized_img_512.astype(np.float32)
#     # if target_val.max() > 1.0: target_val /= 255.0
#     # target_tensor = torch.tensor(target_val, device=device)
# else:
#     target_tensor = resized_img_512.to(device)

# Compute aerial/resist for target
resist_initial_tensor, aerial_initial_tensor = compute_aerial_img(resized_img_512, NA, wavelength, nm_per_pixel)

# Compute aerial/resist for optimized mask
# Ensure val_p is on device
val_p_tensor = val_p.to(device)
resist_optimized_tensor, aerial_optimized_tensor = compute_aerial_img(final_mask, NA, wavelength, nm_per_pixel)

# Move to CPU for plotting
aerial_initial_np = aerial_initial_tensor.detach().cpu().numpy()
aerial_optimized_np = aerial_optimized_tensor.detach().cpu().numpy()

fig, ax = plt.subplots(1, 4, figsize=(20, 5))

ax[0].imshow(resized_img_512, cmap='gray')
ax[0].set_title('Target Mask (Original)')

ax[1].imshow(final_mask, cmap='gray')
ax[1].set_title('Optimized Mask')

ax[2].imshow(resist_optimized_tensor.detach().cpu().numpy() , cmap='gray')
ax[2].set_title('Aerial Output (Optimized)')

ax[3].imshow(resist_initial_tensor.detach().cpu().numpy(), cmap='gray')
ax[3].set_title('Initial Aerial Output')
```

```
plt.tight_layout()
plt.show()

print(f"Total runtime: {time.time() - start_time:.2f} seconds")
```




```
Total runtime: 0.52 seconds
```

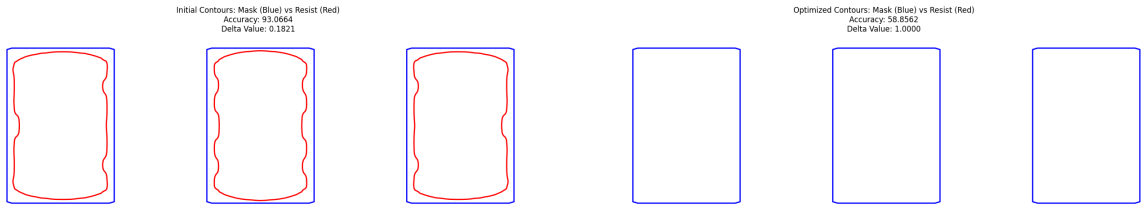
```
def calculate_accuracy_general(mask_np, resist_tensor, threshold=0.25):
    # Convert resist_tensor to numpy if it's a torch.Tensor
    if isinstance(resist_tensor, torch.Tensor):
        resist_np = resist_tensor.detach().cpu().numpy()
    else:
        resist_np = resist_tensor

    # Normalize resist if its max value > 1 (assume 0-255 scale)
    if resist_np.max() > 1.0:
        resist_norm = resist_np / 255.0
    else:
        resist_norm = resist_np

    # Binarize resist image for accuracy and precision
    resist_bin = (resist_norm > threshold).astype(np.uint8)
    mask_bin = (mask_np > 0).astype(np.uint8)

    TP = np.sum((mask_bin == 1) & (resist_bin == 1))
    TN = np.sum((mask_bin == 0) & (resist_bin == 0))
    FP = np.sum((mask_bin == 0) & (resist_bin == 1))

    total_pixels = mask_bin.size

    accuracy = ((TP + TN) / total_pixels) * 100 if total_pixels > 0 else 0
    precision = (TP / (TP + FP)) * 100 if (TP + FP) > 0 else 0

    # Calculate delta on normalized resist and mask (both 0-1)
    # Ensure mask_np is also in the 0-1 range for delta calculation
    mask_norm = mask_np / mask_np.max() if mask_np.max() > 0 else mask_np

    abs_diff = np.abs(resist_norm - mask_norm)
    delta_value = np.mean(abs_diff[mask_bin == 1]) if np.sum(mask_bin == 1) > 0 else 0

    return {
        'accuracy': accuracy,
        'precision': precision,
        'delta_value': delta_value
    }
```

```
metrics_initial = calculate_accuracy_general(resized_img_512, resist_initial_tensor)
metrics_optimized = calculate_accuracy_general(resized_img_512, resist_optimized_tensor)
# metrics_binary_predicted = calculate_accuracy_general(resized_img_512, resist_binary_predicted_np)

print("Initial Contour Metrics:", metrics_initial)
print("Optimized Contour Metrics:", metrics_optimized)
# print("Binary Predicted Contour Metrics:", metrics_binary_predicted)
```

```
Initial Contour Metrics: {'accuracy': np.float64(93.06640625), 'precision': np.float64(100.0), 'delta_value': np.f
Optimized Contour Metrics: {'accuracy': np.float64(58.856201171875), 'precision': 0, 'delta_value': np.float64(0.9
```

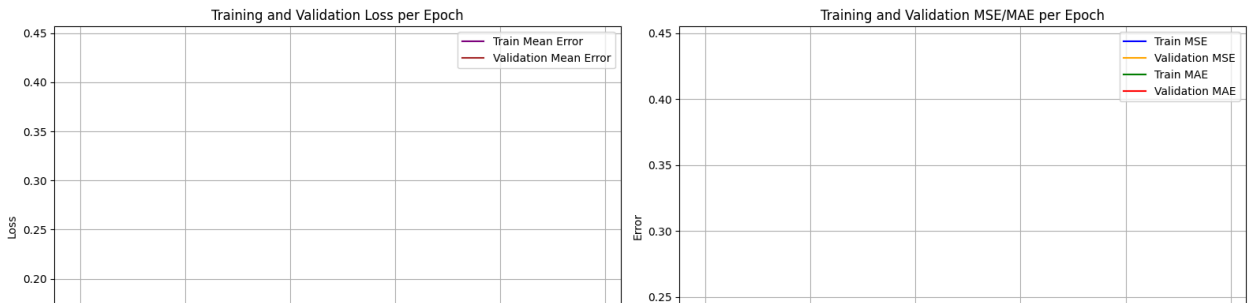
```
# Assuming you have already run the cells above to compute metrics_initial, metrics_optimized, and metrics_binary

# Plotting results
fig, ax = plt.subplots(1, 2, figsize=(28, 5))
```

```
# Initial Contours
# Remove the batch dimension and convert to numpy for plotting
ax[0].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
ax[0].contour(resist_initial_tensor.cpu().numpy(), levels=[0.5], colors='red', linewidths=2)
ax[0].set_title(f"Initial Contours: Mask (Blue) vs Resist (Red)\nAccuracy: {metrics_initial['accuracy']:.4f}\nDe
ax[0].axis('off') # Turn off the axis


# Optimized Contours
ax[1].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
ax[1].contour(resist_optimized_tensor.cpu().numpy(), levels=[0.5], colors='red', linewidths=2)
ax[1].set_title(f"Optimized Contours: Mask (Blue) vs Resist (Red)\nAccuracy: {metrics_optimized['accuracy']:.4f}
ax[1].axis('off') # Turn off the axis

plt.tight_layout()
plt.show()
```




```
# # SAVE + DOWNLOAD (Predicted Mask for test image)
# # ==============================
# pred_np = val_preds[0].detach().cpu().squeeze().numpy()   # (H,W) float [0,1]

# # Option 1: save grayscale probability mask
# pred_img = (pred_np * 255).clip(0, 255).astype(np.uint8)

# # Option 2 (recommended for lithography): save binary mask
# # pred_img = (pred_np > 0.5).astype(np.uint8) * 255

# pred_img = np.ascontiguousarray(pred_img, dtype=np.uint8)

# save_name = f"predicted_test_mask_epoch_{epoch+1}.png"
# cv2.imwrite(save_name, pred_img)

# print("Saved predicted mask in Colab as:", save_name)

# from google.colab import files
# files.download(save_name)
```

```
# # Load the saved model state
# class SingleImageDataset(Dataset):
#         def __init__(self, image_path, transform=None):
#             self.image_path = image_path
#             self.transform = transform

#         def __len__(self):
#             return 1

#         def __getitem__(self, idx):
#             img = Image.open(self.image_path).convert('L')
#             if self.transform:
#                 img = self.transform(img)
#             return img
# model = SmallUNet().to(device)
# model.load_state_dict(torch.load('physics_informed_unet.pth'))
# model.eval()

# # Load the single validation image
# val_image_path = '/content/ILT_contact_120.png'
# val_dataset = SingleImageDataset(
#     image_path=val_image_path,
#     transform=transforms.Compose([
#         transforms.Resize((nx, nx)),
#         transforms.ToTensor(),
#     ])
# )
# val_loader = DataLoader(val_dataset, batch_size=1, shuffle=False)
```

```
# # Get the predicted mask and compute the aerial image
# with torch.no_grad():
#     for val_layout in val_loader:
#         val_layout = val_layout.float().to(device).unsqueeze(1) if val_layout.dim() == 3 else val_layout.to(dev
#         val_preds = model(val_layout)

#         # Compute the aerial image of the predicted mask using the unified function
#         val_aerial, _= compute_aerial_image_torch(val_preds, NA, wavelength, nm_per_pixel)

#         # Display the images
#         show_images(val_layout, val_preds, val_aerial)

#         # Apply threshold to predicted mask to make it binary
#         # binary_predicted_mask = (val_preds > 0.5).float()

#         # Save the predicted mask as an image
#         predicted_mask_np = val_preds.detach().cpu().squeeze().numpy()
#         # Convert to uint8 for saving as an image (scale to 0-255)
#         predicted_mask_img = Image.fromarray((predicted_mask_np * 255).astype(np.uint8), mode='L')
#         save_path = '/content/drive/MyDrive/predicted_mask_contact_120.png'  # Save to Google Drive
#         predicted_mask_img.save(save_path)
#         print(f"Predicted mask saved to: {save_path}")
```

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# First graph: Mean Error (Train & Validation)
axes[0].plot(loss_history['train_mean_error'], label='Train Mean Error', color='purple')
axes[0].plot(loss_history['val_mean_error'], label='Validation Mean Error', color='brown')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Training and Validation Loss per Epoch')
axes[0].legend()
axes[0].grid(True)

# Second graph: MSE and MAE (Train & Validation)
axes[1].plot(loss_history['train_mse'], label='Train MSE', color='blue')
axes[1].plot(loss_history['val_mse'], label='Validation MSE', color='orange')
axes[1].plot(loss_history['train_mae'], label='Train MAE', color='green')
axes[1].plot(loss_history['val_mae'], label='Validation MAE', color='red')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Error')
axes[1].set_title('Training and Validation MSE/MAE per Epoch')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
# Average value of 2nd graph
# after testing avg value of error
```